

AFL ASSISTANT — WEEK 6 DAY 5

Executive Report | Capstone: Full AFL Assistant, Evaluation & Deployment

1. What This Covers

Day 5 hardened, evaluated, wrapped, and documented the existing AFL assistant built across Days 1–4. The system answers AFL factual questions, retrieves exact statistics from a historical dataset, and generates model-based match/player predictions while remaining strictly scoped to AFL topics.

Architecture finding. Inspection revealed two parallel implementations: an incomplete LangGraph state machine (graph.py/nodes.py), including a stub factual node and no wired input-extraction step, and a working LangChain tool-calling agent (agent.py/tools.py). Day 5 focused on hardening the working agent path rather than completing the unused state machine.

2. Hardening Completed

Area	Before	After
Tool errors	Invalid team name (ValueError) or any tool exception could crash the request.	Tool calls are isolated; exceptions and a 10s timeout are caught and converted into normal messages.
Prediction access	AFL_TOOLS registered only the 3 retrieval tools; prediction tools were unreachable.	predict_match and predict_top_players are registered and callable.
Prediction disclaimer	Depended on the LLM remembering to include one.	Programmatically enforced after prediction tool execution if missing.
Abuse handling	None.	Per-conversation sliding-window limit: 20 requests / 60 seconds; message cap: 2,000 characters.
Scope-override resistance	System prompt only.	Added secondary keyword heuristic in guardrails.py as an LLM-independent defense.
Logging	print() statements only.	Structured JSON-line event per turn: query, conversation ID, tool calls, latency, model, status/errors.
API access	Agent callable only from Python.	FastAPI POST /chat, GET /health, plus a minimal optional web UI.

Additional fix. tests/test_guardrails.py referenced guardrail_test.csv instead of guardrail_tests.csv. This filename issue was corrected.

3. Evaluation Results

Evaluation suite. 30 cases across six categories: factual, retrieval, prediction, scope, prompt-injection (5 cases), and multi-turn.

Category / Check	Result	Evidence / Status
Retrieval tool checks	5 / 5 passed	Executed without live LLM access.
Prediction tool checks	5 / 5 passed	Executed without live LLM access.
Prompt-injection heuristic	2 / 5 → 5 / 5	Initial gap found and fixed by widening detection patterns.
Factual Q&A	Pending live review	Requires live LLM call.
Natural-language scope refusals	Partially executed	Requires live LLM; some live cases were affected by API rate limits.
Multi-turn coherence	Pending live review	Requires live LLM call.

Prompt-injection finding. The first heuristic missed persona-override, fake “SYSTEM OVERRIDE” framing, and “just a joke” reframing. These cases were subsequently detected, producing 5/5 on the deterministic heuristic test.

4. Model Comparison

Real model/data result. On a chronological holdout of 1,373 matches, the production Gradient Boosting match-winner model achieved 74.5% accuracy. The dataset contains no ladder-position column, so the comparison uses the closest available proxies rather than a literal ladder-position baseline.

Method	Holdout Accuracy (n=1,373)
Gradient Boosting (production model)	74.5%
Always predict home win	57.2%
Head-to-head win-rate proxy	53.0%
Recent-form proxy	56.2%

Interpretation. The trained model exceeds these simple baselines by approximately 18–22 percentage points, indicating that it captures more than home-field advantage, head-to-head history, or recent form alone.

5. Weakest Category & Improvement

Weakest confirmed area: prompt-injection resistance at the heuristic layer before the fix, measured at 2/5.

Concrete improvement: replace the hand-maintained keyword list with a small dedicated classifier call or a temperature-0 LLM check that classifies whether a message is attempting to override AFL scope. This should improve generalization to unseen phrasings.

6. Known Limitations

- Live-LLM-dependent factual, scope, and multi-turn evaluation is not fully complete because the API quota was exhausted; those results must not be presented as measured pass rates.
- The rate limiter and tool executor are in-memory and single-process, so they do not coordinate across multiple server instances.
- The abandoned `graph.py/nodes.py` state-machine path remains in the codebase and should eventually be completed or formally retired.
- `config.py` currently wires up OpenRouter; the previously discussed NetixSol tiered fallback remains commented out and was not restored during Day 5.
- No automated retraining pipeline exists. The weekly refresh loop is documented as a process rather than automated.

7. Recommended Next Steps

1. Run the remaining live evaluation cases when a working API quota/key is available and record the actual factual, scope, and multi-turn outcomes.
2. Replace the keyword-based injection heuristic with a cheap classifier-based check.
3. Decide whether NetixSol, OpenRouter, or a tiered combination is the intended production LLM path and align `config.py` and `.env.example`.
4. Either finish or formally retire `graph.py/nodes.py` to remove ambiguity about the active architecture.
5. If usage grows beyond one process, move rate limiting to a shared store such as Redis.